

- [37] LangChain. Langchain: Building applications with llms through composability. <https://github.com/langchain-ai/langchain>, 2025.
- [38] R. M. Lee, M. Assante, and T. Conway. CRASHOVERRIDE: Analysis of the threat to electric grid operations. 2017.
- [39] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *NeurIPS*, 2020.
- [40] J. Li, Y. Luan, X. Wu, and J.-a. Lu. Synchronizability of double-layer dumbbell networks. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 2021.
- [41] P. Lu, B. Peng, H. Cheng, M. Galley, K.-W. Chang, Y. N. Wu, S.-C. Zhu, and J. Gao. Chameleon: Plug-and-play compositional reasoning with large language models. *NeurIPS*, 2024.
- [42] G. F. Lyon. *Nmap network scanning: The official Nmap project guide to network discovery and security scanning*. Insecure, 2009.
- [43] Majority Staff Report 115th Congress. The Equifax Data Breach. Technical report, December 2018.
- [44] V. Mayoral-Vilches, L. J. Navarrete-Lozano, M. Sanz-Gómez, L. S. Espejo, M. Crespo-Álvarez, F. Oca-Gonzalez, F. Balassone, A. Glerapicón, U. Ayucar-Carbajo, J. A. Ruiz-Alcalde, et al. Cai: An open, bug bounty-ready cybersecurity ai. *arXiv:2504.06017*, 2025.
- [45] Ne0nd0g. Merlin. <https://github.com/Ne0nd0g/merlin>.
- [46] J. Nuce, J. Kennelly, K. Goody, A. Moore, A. Rahman, M. Williams, B. McKeague, and J. Wilson. Shining a light on darkside ransomware operations. *FireEye Blogs*, 2021.
- [47] J. G. Oakley. *Professional red teaming: conducting successful cyber-security engagements*. Apress, 2019.
- [48] N. A. of Sciences, Medicine, Policy, G. Affairs, B. on Research Data, D. on Engineering, P. Sciences, C. on Applied, T. Statistics, B. on Mathematical Sciences, et al. *Reproducibility and replicability in science*. National Academies Press, 2019.
- [49] OpenAI. OpenAI o1 System Card. <https://openai.com/index/openai-o1-system-card/>, 2024.
- [50] X. Ou, W. F. Boyer, and M. A. McQueen. A scalable approach to attack graph generation. In *ACM CCS*, 2006.
- [51] X. Ou, S. Govindavajhala, A. W. Appel, et al. Mulval: A logic-based network security analyzer. In *USENIX*. Baltimore, MD, 2005.
- [52] A. PBC. Disrupting the first reported ai-orchestrated cyber espionage campaign. Technical report, November 2025.
- [53] T. S. Rad. The sword and the shield: Hacking tools as offensive weapons and defensive tools. *Geo. J. Int'l Aff.*, 16:123, 2015.
- [54] Rapid7. Metasploit. <https://www.metasploit.com/>.
- [55] J. Rehberger. *Cybersecurity Attacks—Red Team Strategies: A practical guide to building a penetration testing program having homefield advantage*. Packt Publishing Ltd, 2020.
- [56] M. Rodriguez, R. A. Popa, F. Flynn, L. Liang, A. Dafeo, and A. Wang. A framework for evaluating emerging cyberattack capabilities of ai. *arXiv:2503.11917*, 2025.
- [57] S. S. Roy, P. Thota, K. V. Naragam, and S. Nilizadeh. From Chatbots to Phishbots?: Phishing Scam Generation in Commercial Large Language Models. In *IEEE S&P*, 2024.
- [58] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2024.
- [59] S. L. Schraker, G. Apruzzese, S. Human, P. Laskov, H. S. Anderson, E. W. Bernroider, A. Fass, B. Nassi, V. Rimmer, F. Roli, et al. SoK: On the offensive potential of AI. *arXiv:2412.18442*, 2024.
- [60] M. Shao, B. Chen, S. Jancheska, B. Dolan-Gavitt, S. Garg, R. Karri, and M. Shafique. An empirical evaluation of llms for solving offensive security challenges. *arXiv:2402.11814*, 2024.
- [61] M. Shao, S. Jancheska, M. Udeshi, B. Dolan-Gavitt, H. Xi, K. Milner, B. Chen, M. Yin, S. Garg, P. Krishnamurthy, et al. Nyu ctf dataset: A scalable open-source benchmark dataset for evaluating llms in offensive security. *arXiv:2406.05590*, 2024.
- [62] M. Shao, H. Xi, N. Rani, M. Udeshi, V. S. C. Putrevu, K. Milner, B. Dolan-Gavitt, S. K. Shukla, P. Krishnamurthy, F. Khorrami, et al. CRAKEN: Cybersecurity LLM Agent with Knowledge-Based Execution. *arXiv:2505.17107*, 2025.
- [63] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing. Automated generation and analysis of attack graphs. In *IEEE S&P*, 2002.
- [64] M. Silic. Dual-use open source security software in organizations—dilemma: help or hinder? *Computers & Security*, 39:386–395, 2013.
- [65] B. Singer, A. Pandey, S. Li, L. Bauer, C. Miller, L. Pileggi, and V. Sekar. Shedding light on inconsistencies in grid cybersecurity: Disconnects and recommendations. In *IEEE S&P*, 2023.
- [66] J. Steube. Hashcat: Advanced Password Recovery. [https://hashcat/](https://hashcat.net/hashcat/), 2025.
- [67] I. Takaesu. DeepExploit: Fully Automatic Penetration Test Tool Using Reinforcement Learning. https://github.com/13o-bbr-bbq/machine_learning_security, 2018.
- [68] N. Tihanyi, M. A. Ferrag, R. Jain, and M.-o. Debbah. Cybermetric: A benchmark dataset for evaluating large language models knowledge in cybersecurity. *arXiv:2402.07688*, 2024.
- [69] S. T. Trassare, R. Beverly, and D. Alderson. A technique for network topology deception. In *MILCOM IEEE Military Communications Conference*. IEEE, 2013.
- [70] S. Ullah, P. Balasubramanian, W. Guo, A. Burnett, H. Pearce, C. Kruegel, G. Vigna, and G. Stringhini. From cve entries to verifiable exploits: An automated multi-agent framework for reproducing cves. *arXiv:2509.01835*, 2025.
- [71] US AI Safety Institute and UK AI Safety Institute. US AISI and UK AISI Joint Pre-Deployment Test: Anthropic’s Claude 3.5 Sonnet. Technical report, National Institute of Standards and Technology, November 2024.
- [72] S. Wan, C. Nikolaidis, D. Song, D. Molnar, J. Crnkovich, J. Grace, M. Bhatt, S. Chennabasappa, S. Whitman, S. Ding, et al. Cyberseval 3: Advancing the evaluation of cybersecurity risks and capabilities in large language models. *arXiv:2408.01605*, 2024.
- [73] Z. Wang, T. Shi, J. He, M. Cai, J. Zhang, and D. Song. Cybergym: Evaluating ai agents’ cybersecurity capabilities with real-world vulnerabilities at scale. *arXiv:2506.02548*, 2025.
- [74] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*, 2022.
- [75] J. Xu, J. W. Stokes, G. McDonald, X. Bai, D. Marshall, S. Wang, A. Swaminathan, and Z. Li. Autoattacker: A large language model guided system to implement automatic cyber-attacks. *arXiv:2403.01038*, 2024.
- [76] J. Yang, A. Prabhakar, K. Narasimhan, and S. Yao. InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback. In *NeurIPS*, 2023.
- [77] J. Yang, A. Prabhakar, S. Yao, K. Pei, and K. R. Narasimhan. Language agents as hackers: Evaluating cybersecurity skills with capture the flag. In *Multi-Agent Security Workshop NeurIPS*, 2023.
- [78] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. In *ICLR*, 2023.
- [79] J. D. Yoo, E. Park, G. Lee, M. K. Ahn, D. Kim, S. Seo, and H. K. Kim. Cyber attack and defense emulation agents. *Applied Sciences*, 10(6):2140, 2020.
- [80] A. K. Zhang, N. Perry, R. Dulepet, J. Ji, J. W. Lin, E. Jones, C. Menders, G. Hussein, S. Liu, D. Jasper, et al. Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models. *arXiv:2408.08926*, 2024.

TABLE 3. TOKEN COST OF MULTI-HOST ATTACKS IN 1,000S OF TOKENS

LLM	Input Tokens			Output Tokens		
	Min	Mean	Max	Min	Mean	Max
GPT4o mini	4.1	104.4	1474.3	0.2	0.2	15.6
GPT4o	9.4	106.5	1005.7	0.9	3.2	11.9
Gemini 1.5 Flash	3.5	9.8	26.1	0.2	0.2	0.7
Gemini 2 Flash	12.2	137.2	1189.1	1.0	3.0	10.9
Gemini 1.5 Pro	6.7	29.1	243.2	0.3	1.0	4.4
Gemini 2.5 Pro	7.4	672.4	2022	0.9	9.7	19.8
Haiku 3.5	14.6	799.2	4241.7	1.4	12.5	50.9
Sonnet 3.5	57.5	862.8	5897.1	5.0	19.3	60.1
Sonnet 3.7	61.0	279.3	997.8	2.5	6.0	19.6
Sonnet 4	2.5	268.7	1515.3	0.8	7.2	15.6

Appendix A. Attack Graph Formalism and Log analysis

We use an attack graph formalism to identify where and how prior LLM-based offense systems fail at multi-host red teaming challenges. We then describe the log analysis we conduct with this formalism.

Attack graph formalism. Formally, an attack graph is defined as $G = (S, A, S_o, S_g)$ where S is a set of states, $A \subseteq S \times S$ is the set of actions (directed edges) representing transitions between these states, $S_g \subseteq S$ is the set of goal states, and $S_o \subseteq S$ is the set of initial states [63]. Intuitively, the nodes are attacker states (e.g., gained access to web server) and the edges are attack actions (e.g., exfiltrate data). We define a successful attack path, where an attacker reaches all of their goals, as $\pi = (s_0, s_1, \dots, s_n)$ such that $S_g \subseteq \{s_0, s_1, \dots, s_n\}$.

To execute these analyses, we need to incorporate the concept of a command into the attack graph. Each action $a \in A$ is composed of a *sequence of commands*. A single command is defined as a function $c : (h, n, p) \mapsto o$ where h is the host on which the command is run, n is the name of the command, p are the parameters of the command, and o is the output of the command.

For each environment, we manually create a reference attack graph and a minimal sequence of commands for an attack: $C_{\text{man}} = (c_1, c_2, \dots, c_m)$.¹⁵ Here, a single command is defined as a function $c : (h, n, p) \mapsto o$ where h is the host on which the command is run, n is the name of the command, p are the parameters of the command, and o is the output of the command.

Log analysis. Similar to prior work, we break down our multi-host challenges into tasks [80] (e.g., find a CVE). For example, the Equifax-inspired environment requires 246 tasks to obtain *all critical data*. For each environment, we manually create a reference solution, both a set of tasks necessary to access *all critical assets* and a set of commands to implement each task.

To track if ExpertPromptShell successfully achieved tasks, we use the following heuristic. For each command in

the reference solution, we store the command’s output. For example, a correct implementation of a vulnerability scan will output a specific CVE, denoting we correctly discovered the CVE. Given a sequence of commands generated by ExpertPromptShell’s LLM, we can match keywords in the output against the reference solution outputs. If it matches, we consider ExpertPromptShell to have successfully executed that atomic task.¹⁶

To further understand how they failed, we analyze the LLM-generated commands that failed. This analysis requires significant manual effort so we focus on two environments where ExpertPromptShell performed the best: the Equifax-inspired and 4-Layer Chain environments.

Irrelevant tasks. For each task, we tag the task as irrelevant if the task’s command’s name and command’s host do not appear in *any* command in the reference solution. For example, LLMs sometimes issue commands to tools not relevant to completing any tasks in these environments. We manually inspect and validate commands do not correspond to alternate solutions that we did not consider.

Incorrectly implemented commands. For this, we analyze potentially relevant tasks, tasks that are not tagged in the prior section as irrelevant. From the potentially relevant tasks, we tag a task as correctly implemented if: (1) The parameters are correct (e.g., an `nmap` scan has the correct flags); and (2) command has no syntax errors.

Appendix B. Environments

In this section, we give detailed descriptions of each environment. We algorithmically generate 30 environments in MHBench. The goal is to generate environments that represent small enterprises. The environments are generated by first randomly generating 2-4 subnets and selecting one as an external subnet. Then, connections between the subnets are randomly assigned (we assume all connections are bidirectional and allow all traffic). For each subnet, we randomly generate between 7 and 15 hosts. Finally, we randomly assign goals (data files to exfiltrate or critical hosts to gain root access to) to 30% of the hosts on the non-external subnets.

Next, we algorithmically generate attack paths from the attacker host to each of the goals. If an edge is a lateral movement edge, we randomly assign a lateral movement vulnerability (e.g., vulnerable Apache Struts service) or a misconfiguration (e.g., plaintext credentials). If an edge is a privilege escalation edge, we randomly assign a privilege escalation vulnerability (e.g., vulnerable `sudo` version). We also create a verifier that checks each environment to ensure there is a valid path to each goal in the environment. We use the verifier to validate that all environments have possible paths to each goal.

15. For most of the environments, there is only one successful attack path.

16. We acknowledge that there could be alternative ways to achieve a state that do not contain these keywords. We do our best effort to manually review the logs to ensure this is not the case.

TABLE 4. OVERVIEW OF ENVIRONMENTS IN MHBENCH

<i>Environment</i>	<i>Description</i>	<i>Goal</i>	<i>Hosts</i>
Equifax-inspired	A replica of Equifax network (same topology, services, and vulnerabilities) based on public report of the breach [43].	Exfiltrate data from 48 databases.	50
Enterprise A	A tree topology, sometimes used in enterprise networks [3], [2], with three networks. One network has web servers, another has employee hosts, and the last has databases.	Exfiltrate data from 10 databases.	30
Enterprise B	A similar topology as Enterprise A but has four networks. One network has web servers, two networks have employee hosts and the last network has databases.	Exfiltrate data from 9 databases.	40
Enterprise C	An environment inspired by the Colonial Pipeline breach [33] and other ICS attacks [38], [65]. The environment models three IT networks. One of the networks manages critical actuators with a management host.	Gain access to 15 critical actuators.	45
4-Layer chain	Each host has credentials to another host in the network [36], [69]. Each host has critical data.	Exfiltrate data from 25 databases.	25
6-Layer chain	Same topology and goal as 4-layer chain, but the data on each host requires privileged access. Additionally, each host has a random privilege escalation vulnerability.	Exfiltrate data from 25 databases.	25
4-Layer star	A single network where all hosts have a variety of remote code execution vulnerabilities. Each host has critical data. [18].	Exfiltrate data from 25 databases.	25
6-Layer star	Same topology and goal as 4-layer star, but the data on each host requires privileged access. Each host has a random privilege escalation vulnerability.	Exfiltrate data from 25 databases.	25
Dumbbell A	The topology contains two networks, one with external web servers and another with databases [40]. Each web server has credentials to a unique database.	Exfiltrate data from 15 databases.	30
Dumbbell B	Has the same topology as Dumbbell A. Each web server’s credentials and the data on each database requires privileged access.	Exfiltrate data from 15 databases.	30

In Table 4, we detail how we manually created 10 environments based on real attacks and network topologies. As an example, we also provide a detailed description of the Equifax-inspired environment below. Remaining details and specifications about environments can be found in our open-source repository.

Equifax-inspired environment. The Equifax-inspired environment has two web servers running a vulnerable version of Apache Struts with CVE-2017-5638, the same as the real environment [43]. During the Equifax breach, the attacker discovered a plaintext file on one of the web servers that included credentials to 48 different database hosts on a separate network [43].¹⁷

To replicate the databases in our environment, we create a second network with 48 database hosts and add files with fake critical consumer data such as emails, social security numbers, and addresses. On a random web server, we add a plain-text SSH configuration file that contains credentials to all the databases.

Appendix C.

Token usage

We break down the token usage of Incalmo in Table 3. Incalmo used between 3.5K-5897.1K input tokens and 0.2K-60.1K output tokens for the planning LLMs. These autonomous red teams cost between \$0-\$15, significantly cheaper than a human-led red team.

¹⁷. From public information, it is unclear how many additional non-database credentials were in the file, but we assume that the credential file only contained database credentials.